

ELI — Cross-Platform Smoke-Test Checklist

Contents

Part 0 — What’s already validated (don’t re-test)	1
Part 1 — Environment pre-flight (per OS)	1
Part 2 — Install (per OS)	2
Part 3 — First run & the wizard	2
Part 4 — Core actions (the “does it actually work” 8)	2
Part 5 — Server + phone/device (any OS)	3
Part 6 — OS-specific risk areas (watch these — most likely to differ)	3
Scoring & what to do with failures	4

A ~10-minute-per-OS validation that a fresh GitHub clone installs and runs correctly on a different machine/OS/device. Run it on each target (Linux / macOS / Windows) and tick the boxes. Anything that fails is a real, reproducible bug to file.

Part 0 — What’s already validated (don’t re-test)

These were verified against the fresh clone (git archive HEAD = exactly what GitHub serves) and are **sound by construction** — the smoke test confirms them *on the target OS*, not from scratch:

- **Fresh download is clean:** 19 MB / 834 files, zero personal data, zero secrets, zero user DBs; models/ present (.gitkeep).
- **Package builds:** eli_mkxi-2.0.0-py3-none-any.whl builds cleanly from the clone.
- **pyproject.toml complete:** name/version/requires-python >=3.10, 18 core deps, 14 extra groups, [full] meta-extra (eli_mkxi[gui,tts,asr,llm,image,docs,server,vision,web,...]), 3 console scripts (eli/eli_mkxi/eli_cli → eli.gui.app:main), setuptools backend.
- **Dependencies covered across 4 sources:** core deps + .[full] extras (PySide6, faster-whisper, piper, fastapi, transformers, opencv...) + requirements.lock.txt (160 pinned) + **explicit torch & llama-cpp** (GPU-variant, installed by the OS installer).
- **Installers exist per-OS:** install.sh (Linux/macOS), install.ps1 + install.bat (Windows), install_android.sh; per-OS requirements-{windows,macos}.txt.
- **Config/data dirs resolve per-OS:** XDG (Linux) / APPDATA+LOCALAPPDATA (Windows) / Library (macOS).
- **First-run creates a blank slate:** init_data builds all 4 DBs, “no personal data written”; config is offline-by-default (network_enabled: false).

Confidence going in: Linux [yes] fully validated by the author’s runs. macOS/Windows [intermediate] built for it, not yet executed — that’s exactly what this checklist closes.

Part 1 — Environment pre-flight (per OS)

#	Check	How	Pass
1.1	Python ≥ 3.10	python3 --version (Win: python --version)	☐

#	Check	How	Pass
1.2	git clone succeeds	git clone <repo> && cd ELI_v2.0	☐
1.3	Disk ≥ 10 GB free (model + deps)	installer prints it	☐
1.4	(GPU box) driver present	nvidia-smi / macOS Metal is automatic	☐

Part 2 — Install (per OS)

#	Check	How	Pass
2.1	Installer runs to completion	Linux/macOS: bash install.sh · Windows: powershell -ExecutionPolicy Bypass -File install.ps1	☐
2.2	venv created	.venv/ exists	☐
2.3	import eli works in the venv	.venv/bin/python -c "import eli" (Win: .venv\Scripts\python)	☐
2.4	Console script installed	eli --help or .venv/.../eli resolves	☐
2.5	GPU offload verified (GPU boxes)	installer prints “llama-cpp has CUDA/Metal GPU offload” (or an honest CPU-only warning)	☐
2.6	Config seeded	config/settings.json created from template, network_enabled: false	☐
2.7	DB architecture built	installer prints “Full database architecture ready (blank slate)”	☐
2.8	Embedder downloaded	installer prints “Embedder ready” (or a deferred note)	☐

Part 3 — First run & the wizard

#	Check	How	Pass
3.1	App launches	desktop: scripts/eli_launch.sh / the shortcut · GUI window opens	☐
3.2	Setup wizard appears (first launch only)	3 short questions; Skip also works	☐
3.3	Model present or offered	wizard offers a download sized to VRAM, or a model is already there	☐
3.4	4 DBs exist under the data dir	user/agent/system_index/coding_memory.sqlite3, empty of personal rows	☐
3.5	Offline by default	no network activity until you ask for news/search/download	☐

Part 4 — Core actions (the “does it actually work” 8)

Type each in Chat; tick if it does the real thing (not a fake confirmation):

#	Say	Expect	Pass
4.1	how are you?	a natural reply in ELI's voice (model loaded)	☐
4.2	what can you do?	capability list	☐
4.3	what time is it?	correct local time	☐
4.4	open firefox (or any installed app)	the app actually opens (OS control)	☐
4.5	what's on my screen?	a real screenshot description (cross-OS capture)	☐
4.6	create a file test.txt then read test.txt	file created + read back (FS + security gate)	☐
4.7	remember my dog is Rufus → what's my dog's name?	recalled (memory write+read)	☐
4.8	set a timer for 1 minute	timer set + fires (scheduling)	☐

Part 5 — Server + phone/device (any OS)

#	Check	How	Pass
5.1	Server starts in LAN mode	python -m api.server --lan --https prints the phone URL + #token=	☐
5.2	Phone opens the dashboard	scan the QR / open the URL on a phone on the same Wi-Fi	☐
5.3	Token survives a restart	restart the server; the same phone link still works (stable token)	☐
5.4	Phone mic (HTTPS)	accept the one-time cert warning; voice input works	☐
5.5	Firewall hint shown	if the phone can't reach it, the printed OS-specific firewall command fixes it	☐

Part 6 — OS-specific risk areas (watch these — most likely to differ)

The **embodiment layer** is developed on Linux; it has cross-platform code but exercise it:

#	Area	Windows note	macOS note	Pass
6.1	Screenshot	PIL ImageGrab path	screencapture — may need Screen-Recording permission	☐
6.2	Mouse/keyboard control	pyautogui	pyautogui — needs Accessibility permission	☐
6.3	Window control (open/focus/tile)	pyautogui/pygetwindow	limited; note what works	☐
6.4	Voice out (TTS)	Piper voice loads	Piper voice loads	☐
6.5	Wake word ("computer")	mic access	mic access	☐

#	Area	Windows note	macOS note	Pass
6.6	Media (play X on spotify/youtube)	player detection	player detection	☐

macOS gotcha: first use of screenshot/control will prompt for **Screen Recording** and **Accessibility** permissions (System Settings → Privacy). Grant them, then retry.

Scoring & what to do with failures

- **Parts 1-4 pass** → the OS is **fully usable** for the core assistant. File any Part-4 miss as a blocker.
- **Part 5 passes** → phone/web works on that OS.
- **Part 6 partial** → expected; log exactly which control ops work per OS so the docs can state it honestly (e.g., “window tiling: Linux -, macOS partial, Windows -”).

Report format for each OS: OS + version + arch | Python | GPU | which boxes failed + the error text. That’s enough to reproduce and fix.

Author’s honest note

Linux (x86_64) is validated end-to-end. This checklist exists because macOS and Windows are **built for but not yet executed** by the author — the code has the per-OS branches, installers, and cross-platform libraries, but only a real run on each machine turns [intermediate] into [yes]. Ten minutes per OS closes that gap.